

Temas Selectos de Física Computacional III:

Introducción a la ciencia de datos.

Clase 8: 25 Marzo 2025

Dr. Leonid Serkin

1 Introducción

En esta clase, seguiremos estudiando los arboles de decisión (Decision Trees o DTs), veremos el método de boosting y plantaremos todo un bosque¹ aleatorio utilizando los árboles disponibles en `Scikit-learn`.

2 DTs para el conjunto de datos 2D sintético

1. Carga el conjunto de datos sintético "sintetico_2d.csv". Este conjunto tiene dos características y una etiqueta de clase. Utiliza la función `scatter()` de `matplotlib` para graficar el conjunto de datos en un plano 2D y colorea los puntos en función de su etiqueta de clase.
2. Construye un árbol de decisión simple utilizando el conjunto de datos completo y luego visualiza el modelo aprendido. Qué está aprendiendo el árbol y por qué?
3. Los árboles de decisión de `sklearn` almacenan, para cada división que realizan, la información sobre la característica que están usando para la división y el valor umbral usado en la comparación. Puedes encontrar esta información en `clf.tree_.feature` y `clf.tree_.threshold`.
Con esta información, puedes tomar el código de `dibujar_divisiones.py` y graficar las fronteras de decisión (es decir, los umbrales de las características) aplicados por el árbol de decisión en el conjunto de datos. Idealmente, deberías tener un gráfico 2D con líneas verticales y horizontales que dividan el plano en subregiones (una para cada hoja del árbol).
4. Identifica una **nueva variable muy simple** que se podría construir de las variables de entrada que permitiría al árbol de decisión aprender muy rápido de este problema.

¹Recuerda que cada persona genera aproximadamente 4.5 toneladas de dióxido de carbono al año. Para compensar esa cantidad de CO₂, se necesitan plantar alrededor de 205 árboles.

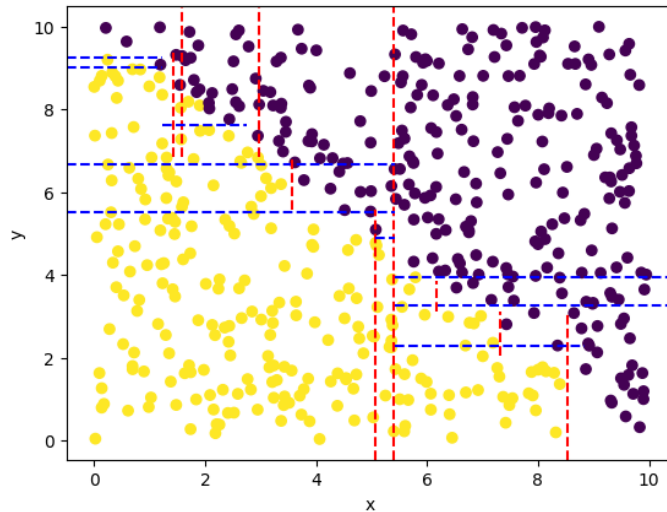


Figure 1: Subregiones para cada hoja de árbol de decisión.

3 DTs para el conjunto de lunas

1. Carga el conjunto de datos sintético de las lunas:

```
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=1000, noise=0.1, random_state=123)
```

Utiliza la función `scatter()` de `matplotlib` para graficar el conjunto de datos en un plano 2D y colorea los puntos en función de su etiqueta de clase.

2. Construye un árbol de decisión simple utilizando este conjunto de datos completo y visualízalo.

```
moon_clf = DecisionTreeClassifier(random_state=123)
```

3. Ahora puedes tomar el código de `dibujar_frontera_decision.py` y graficar las fronteras de decisión
4. Que sucede si cambias la cantidad de ruido (por ejemplo `noise=0.5`) en las lunas?
5. Entrena un `DecisionTreeClassifier`, y compara la precisión en conjuntos de entrenamiento y prueba (80% entrenamiento, 20% test).
6. Demuestra que estamos en un claro caso de sobreajuste (overfitting). Compara `accuracy_score` de conjuntos de entrenamiento y prueba.

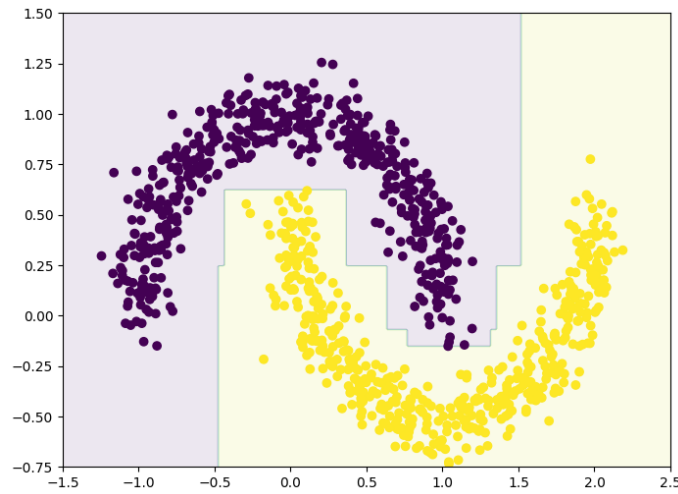


Figure 2: Fronteras de decisión.

4 Gradient Boosting

Gradient Boosting es un método de ensemble que construye modelos de forma secuencial donde cada nuevo árbol se entrena para **corregir los errores del modelo anterior**.

El proceso comienza con un modelo simple, normalmente un árbol pequeño, y luego se agregan nuevos árboles uno por uno. Cada árbol adicional intenta mejorar las predicciones del conjunto ajustando las diferencias (llamadas *residuos*) entre las predicciones anteriores y los valores reales.

Este enfoque se denomina **gradient boosting** porque el modelo minimiza una función de pérdida paso a paso, utilizando el gradiente (dirección de máximo descenso) para reducir el error en cada iteración.

Los parámetros más importantes en **Gradient Boosting** son:

- **n_estimators**: número de árboles que se agregan al modelo.
- **max_depth**: profundidad máxima de cada árbol individual.
- **learning_rate**: controla cuánto contribuye cada árbol nuevo; valores pequeños hacen el aprendizaje más lento pero más preciso.

Tratemos entonces de resolver el problema previo con **Gradient Boosting**. Compara **accuracy_score** de conjuntos de entrenamiento y prueba con el valor de los DTs.

```
from sklearn.ensemble import GradientBoostingClassifier
gb_clf = GradientBoostingClassifier(n_estimators=100, max_depth=5,
learning_rate=0.1, random_state=123)
```

5 Bosques aleatorios

Un bosque aleatorio entrena **varios árboles de decisión sobre diferentes subconjuntos del conjunto de datos**, lo que ayuda a reducir el sobreajuste. Cada árbol se entrena con una muestra de N puntos tomada *con reemplazo* del conjunto original de N ejemplos. Esto significa que algunos puntos pueden repetirse y otros no ser seleccionados. De hecho, en promedio, solo alrededor del 63.2% de los puntos únicos se utilizan en cada árbol.

Además, cada árbol toma decisiones usando solo un subconjunto aleatorio de las características disponibles. Este subconjunto tiene tamaño B , que por defecto suele ser igual a la raíz cuadrada del número total de características. Este valor se puede ajustar mediante el parámetro `max_features`.

Otro parámetro clave es `n_estimators`, que indica cuántos árboles tendrá el bosque. Durante la predicción, cada árbol da su voto, y la clase final se decide por **votación mayoritaria**, es decir, se elige la clase más votada por los árboles.

```
from sklearn.ensemble import RandomForestClassifier
rf_clf = RandomForestClassifier(n_estimators=100, max_depth=5,
                               random_state=123)
```

Crea tu bosque para resolver el problema previo y compara `accuracy_score` de conjuntos de entrenamiento y prueba con el valor de los DTs y los GBDTs.

Para visualizar los resultados, dibuja las fronteras de decisión para los tres casos estudiados.

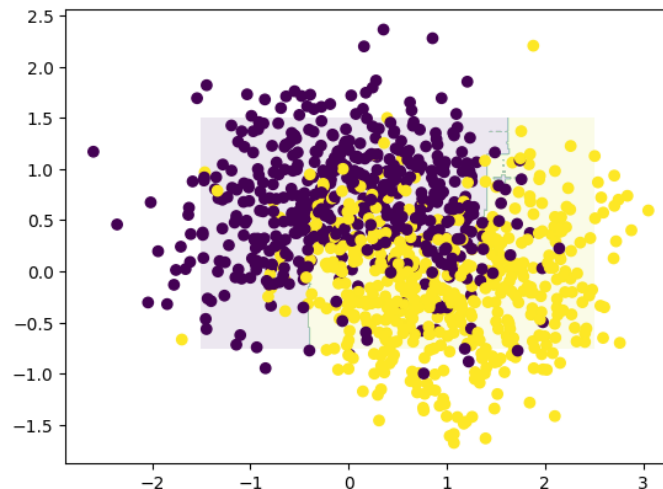


Figure 3: Fronteras de decisión del bosque.

6 Tarea

En esta tarea trabajaremos con el conjunto de datos de cáncer de mama que hemos estudiado anteriormente.

```
from sklearn.datasets import load_breast_cancer
dataset = load_breast_cancer()
X = dataset["data"]
y = dataset["target"]
```

Los pasos a seguir después de cargar el conjunto de datos y dividirlo en un conjunto de entrenamiento (80%) y un conjunto de prueba (20%) son:

1. Entrena un único árbol de decisión usando parámetros razonables (define que es razonable para ti!?) en el conjunto de entrenamiento y luego calcula su precisión en el conjunto de prueba.
2. Evalúa el mismo árbol de decisión utilizando validación cruzada con 5 particiones. Al final, calcula el promedio ponderado de las precisiones obtenidas en cada fold, y compara con el resultado previo.
3. Aplica **gradient boosting** usando parámetros razonables (por ejemplo, **n_estimators=100**, **max_depth=3**) y calcula su precisión en el conjunto de prueba.
4. Analiza cómo afecta el parámetro **learning_rate**, uno de los parámetros más importantes en **gradient boosting**, a la precisión probando diferentes valores (por ejemplo: 0.01, 0.1, 0.3, 0.5).
5. Entrena un bosque aleatorio usando los parámetros que vimos en la clase sobre el conjunto de entrenamiento y calcula su precisión sobre el conjunto de prueba.
6. Compara la matriz de confusión de los tres modelos (árbol, boosting y bosque aleatorio) y comenta sobre los tipos de errores que cometen y cuál generaliza mejor?
7. Estudia cómo varía el rendimiento del bosque aleatorio a medida que crece el número de árboles. Prueba valores de **n_estimators** desde 10 hasta 100 (en pasos de 10), y grafica la precisión en el conjunto de prueba.
8. Al igual que con los árboles de decisión, los bosques aleatorios pueden calcular la importancia de las características utilizadas. Esto se hace agregando la importancia de las características de los árboles en un solo valor. Si I_{ab} es la importancia de la característica a según el árbol b , la importancia de la característica a según el bosque aleatorio se puede calcular como sigue:

$$I_a = \frac{\sum_j I_{aj}}{\sum_i \sum_j I_{ij}}$$

Es decir, la importancia general de una característica se da por la suma de la importancia de esa característica a través de todos los árboles, dividida por la suma de las importancias de todas las características en todos los árboles.

Puedes acceder a esto mediante el atributo `feature_importances_`. Obtén las importancias de las 30 características y visualízalas con una gráfica de barras ordenadas.

9. Las top 10 características coinciden con lo que podrías esperar de su correlación con el diagnóstico (target) que estudiamos en las clases previas? Analiza las más relevantes con una gráfica de dispersión, donde cada punto representa una característica.
10. Investiga y aplica `XGBoost` o `LightGBM` en lugar de `GradientBoostingClassifier`, compara su velocidad de entrenamiento y precisión con los métodos anteriores.

```
import xgboost as xgb
import lightgbm as lgb
GradientBoostingClassifier(n_estimators=10000, max_depth=3, random_state=123))
xgb.XGBClassifier(n_estimators=10000, max_depth=3, eval_metric='logloss',
random_state=123))
lgb.LGBMClassifier(n_estimators=10000, max_depth=3, random_state=123))
```